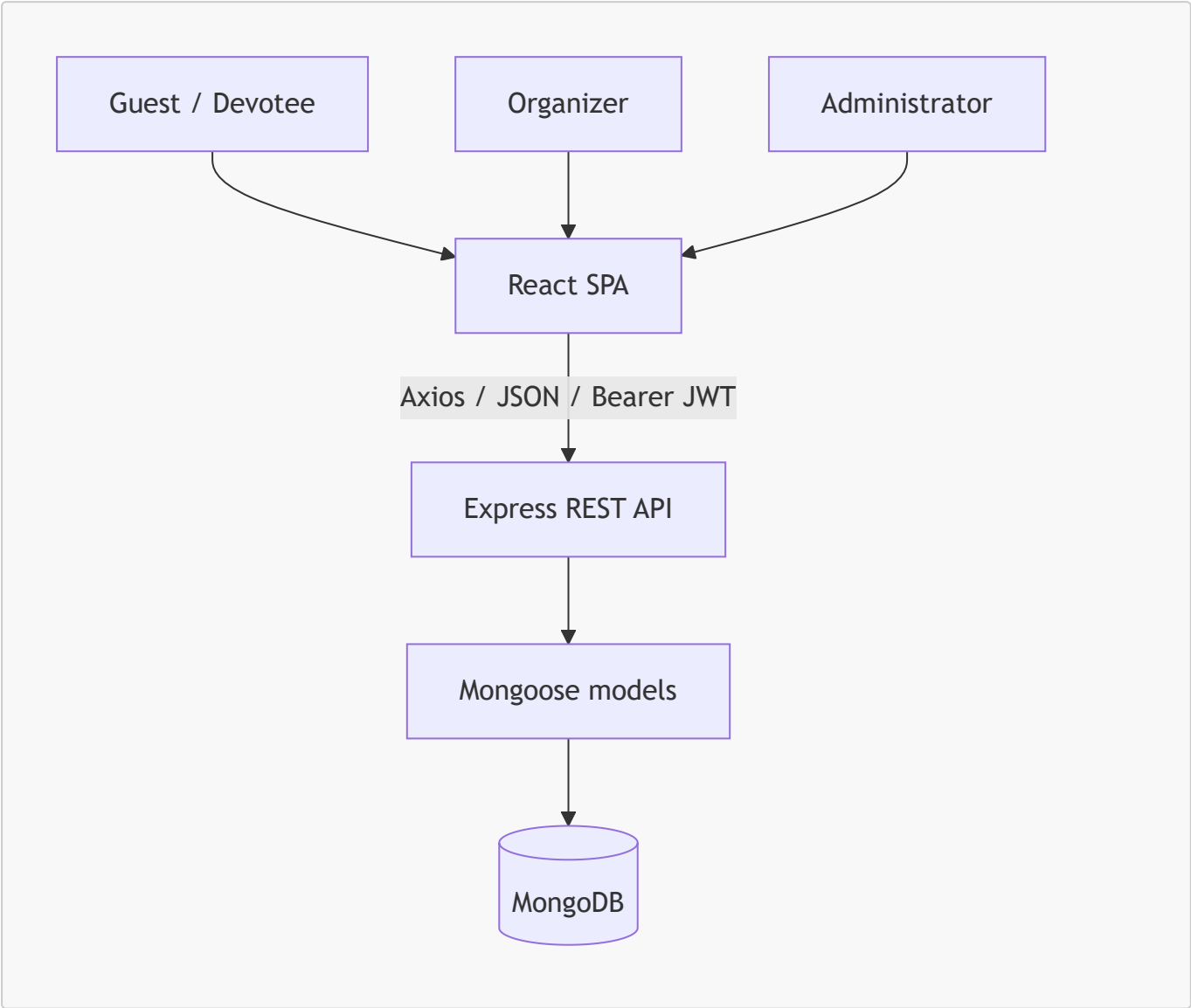


System Architecture

Context



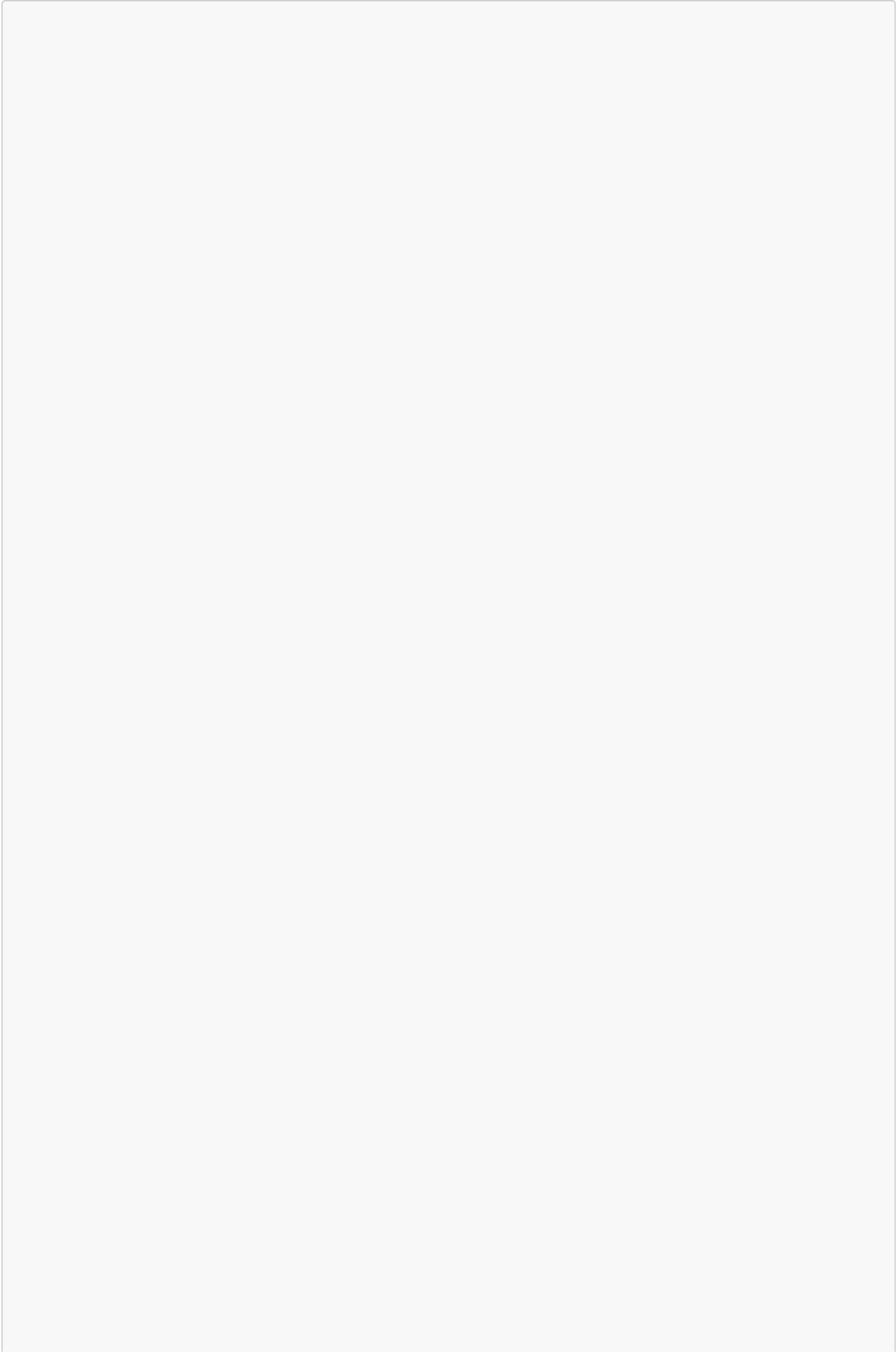
The client is a Vite-built React single-page application. `AuthContext` sets a global Axios base URL and bearer header. Express mounts six API routers, checks database availability for `/api`, applies authentication/role middleware to protected routes, and uses Mongoose models for persistence.

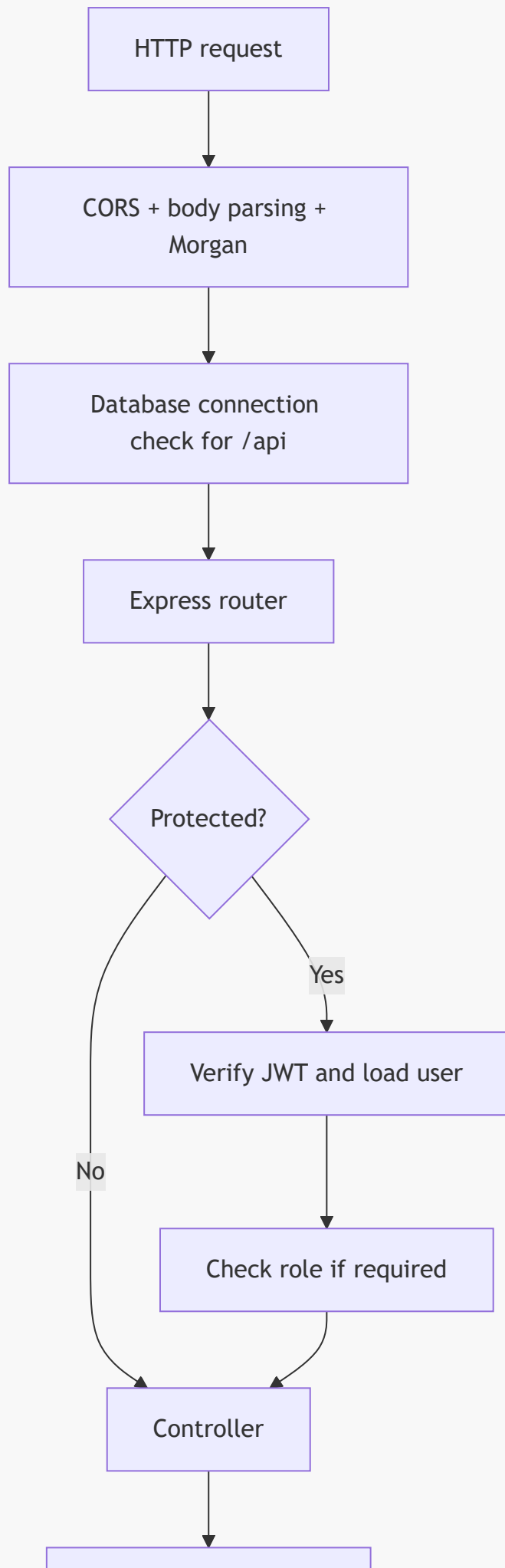
Runtime components

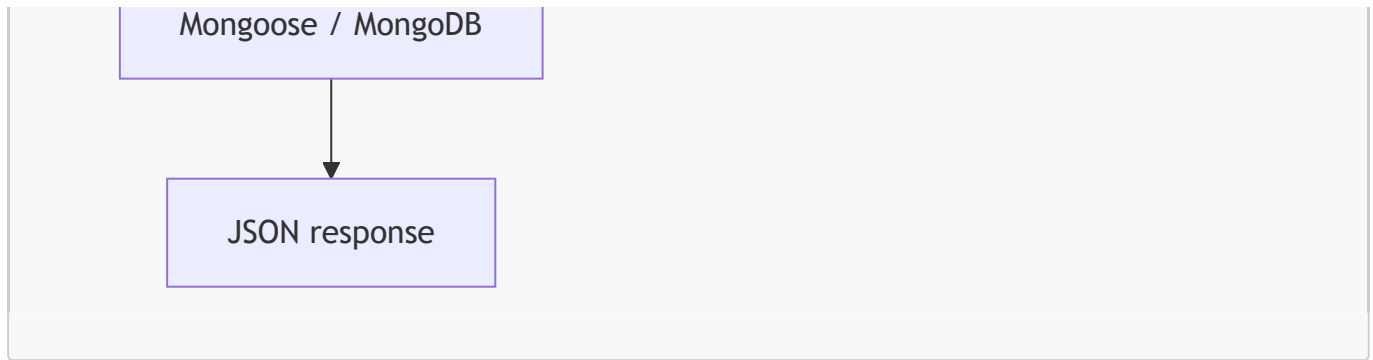
Component	Responsibility	Key source
Router shell	Public pages and three dashboard URLs	<code>client/src/App.jsx</code>
Authentication context	Token storage, Axios configuration, profile lifecycle	<code>client/src/context/AuthContext.jsx</code>

Component	Responsibility	Key source
Audio context/player	Temple devotional audio and mute state	<code>AudioContext.jsx</code> , <code>AudioFloatingPlayer.jsx</code>
Public pages	Discovery, temple detail, login/register, donation	<code>client/src/pages/*</code>
Role dashboards	User history/profile; organizer slots/bookings; admin management/analytics	<code>Dashboard*.jsx</code>
Express server	Middleware, route mounting, health/404/error responses	<code>server/server.js</code>
Controllers	Request validation, business operations, response mapping	<code>server/controllers/*</code>
Auth middleware	JWT verification and role authorization	<code>server/middleware/authMiddleware.js</code>
Mongoose models	User, Temple, DarshanSlot, Booking, Donation schemas	<code>server/models/*</code>
Seeder	Initial temples, events, ratings, and slots	<code>server/config/seeder.js</code>

Backend request pipeline







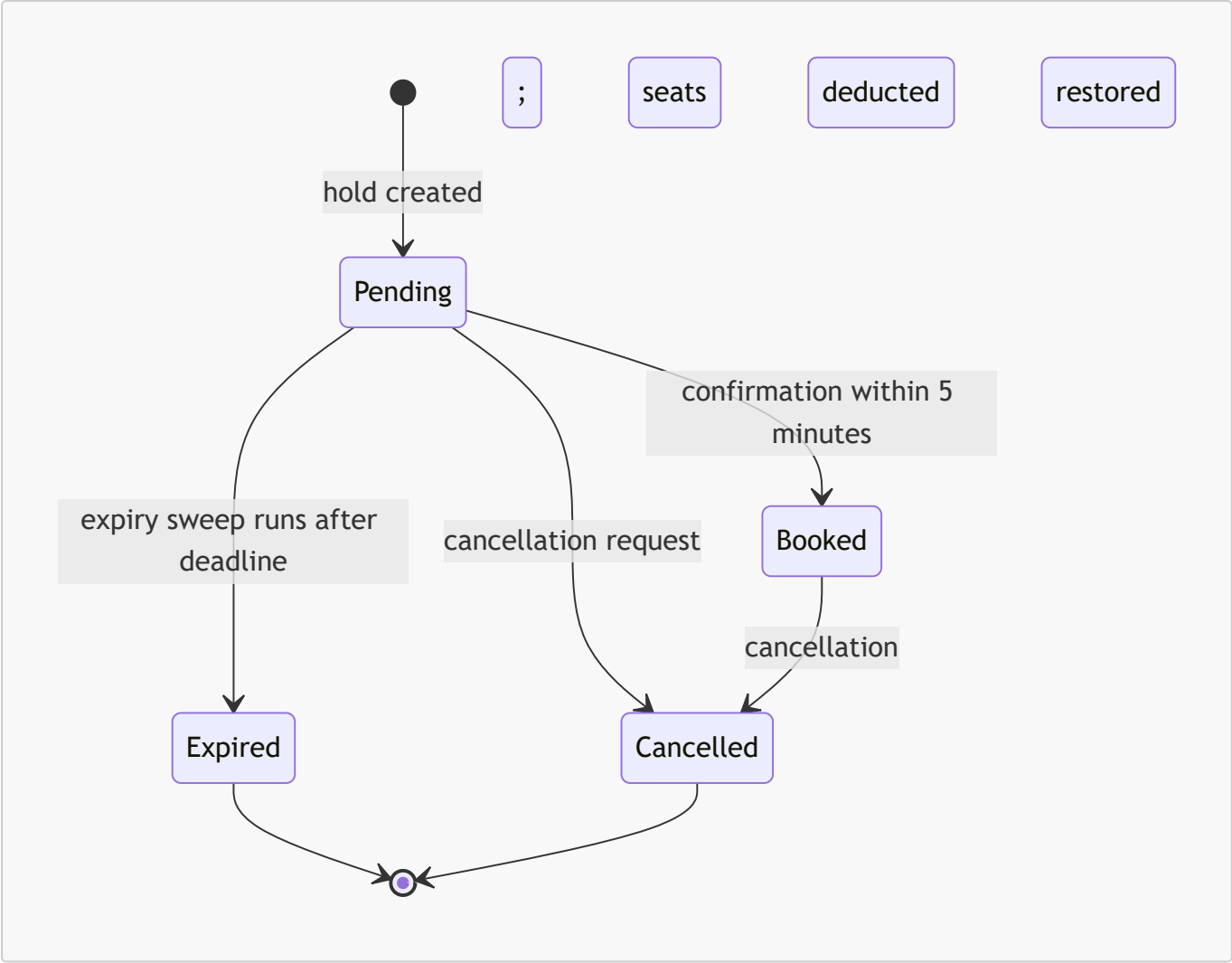
Route authorization matrix

Resource	Guest	USER	ORGANIZER	ADMIN
Read temples/slots	Yes	Yes	Yes	Yes
Review temple	No	Yes	Yes*	Yes*
Create/confirm own booking	No	Yes	API permits**	API permits**
Read own bookings/donations	No	Yes	Yes	Yes
Read all bookings/donations	No	No	Yes	Yes
Create/update/delete slot	No	No	Yes	Yes
Create/update/delete temple	No	No	No	Yes
Events and maintenance admin	No	No	No	Yes
User administration/analytics	No	No	No	Yes

* The review API accepts any authenticated role, although the UI presents reviewing as a user action.

** The API only requires authentication for booking creation/confirmation; the UI restricts slot booking to **USER**. Server-side role enforcement should be added.

Booking lifecycle



The expiry sweep is not a scheduler. It runs only when selected booking or slot controller methods call `releaseExpiredHoldBookings()`. A pending cancellation currently transitions to `Cancelled` without returning the held seats.

Frontend route map

Path	Page	Intended access
/	Home	Public
/temple/:id	Temple details and slots	Public; actions protected
/login	Login	Public
/register	Registration	Public
/donation	Donation	Public view; submission requires login
/dashboard	User dashboard	USER
/organizer	Organizer dashboard	ORGANIZER
/admin	Admin dashboard	ADMIN

There is no `ProtectedRoute` component. Direct dashboard navigation may render UI for the wrong role, but API middleware remains the decisive protection for restricted data/actions.

Design improvement priorities

1. Add server-owned registration roles and explicit organizer-temple assignments.
2. Make seat decrement and booking creation atomic with a conditional update/transaction.
3. Replace lazy expiration with a scheduled job and idempotent state transition.
4. Introduce request DTO validation, centralized async error handling, and typed API contracts.
5. Move the client API URL to an environment variable and configure a production proxy/origin policy.
6. Add route guards for user experience while retaining API authorization.
7. Introduce a real payment adapter only after defining security and compliance boundaries.